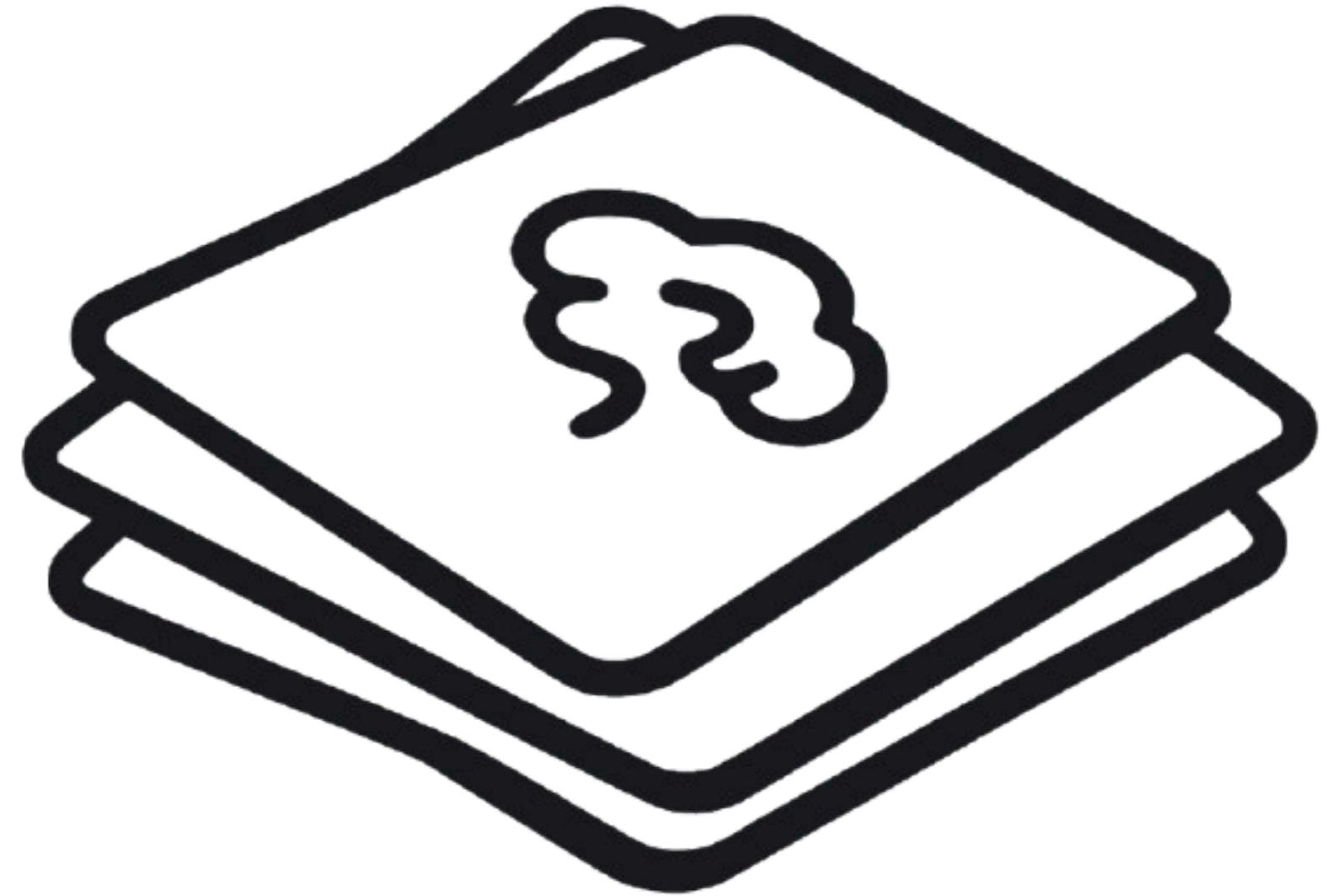


Mars 2026

Présentation de Memoflip

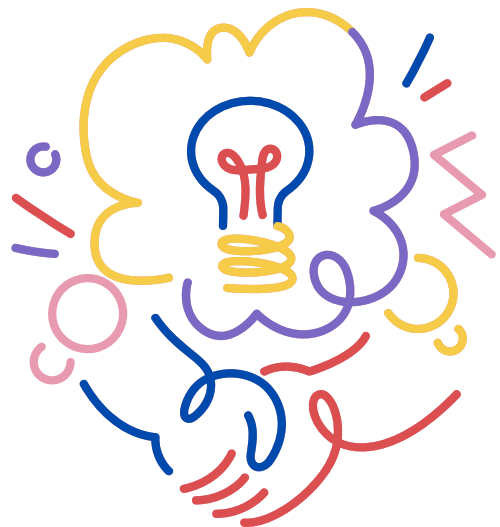
Certification Concepteur Développeur d'Applications



Introduction

Jordan Burdon

Formation CDA - 24/02/2025



Projet

Application web

Fiches de révisions

Quiz complémentaire

Format pratique

Accessible



L'équipe

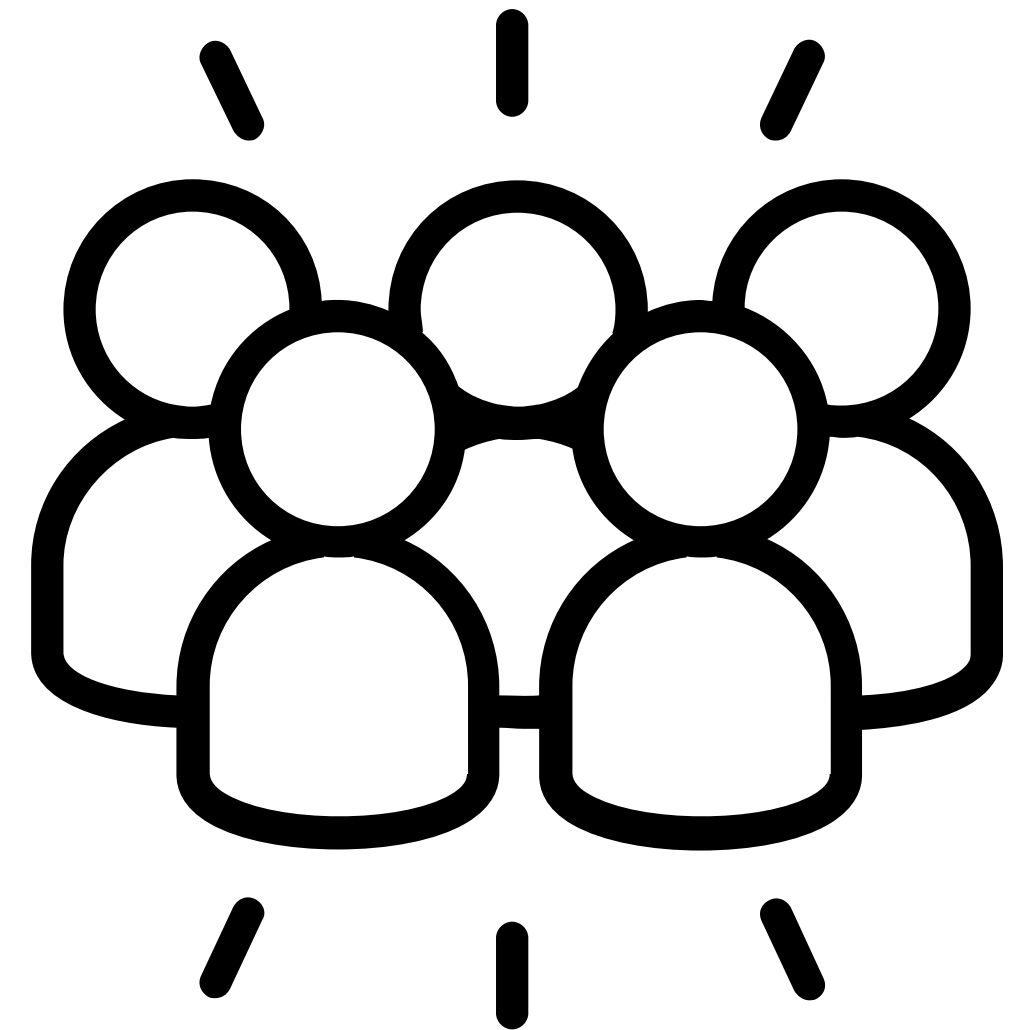
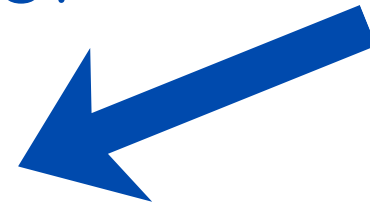
Lead développeur

Product Owner

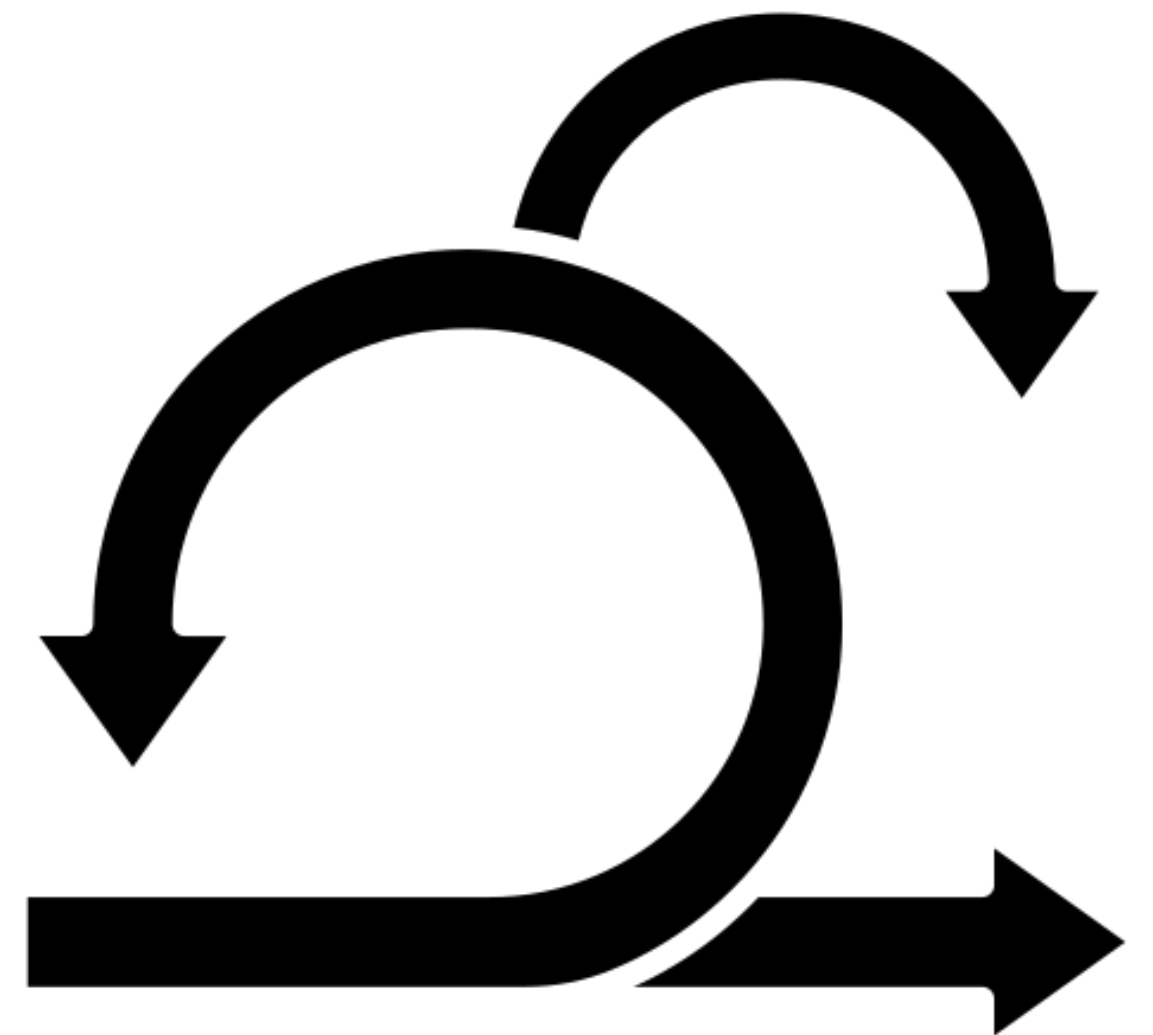
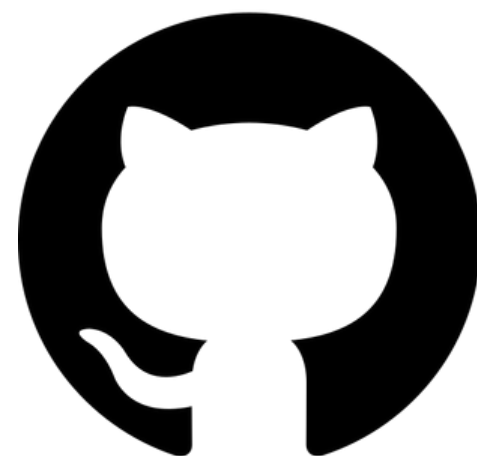
Responsable
Process

UI / UX

Conseiller technique
(formateur)



Communication & Organisation



Stack et Outils

Front



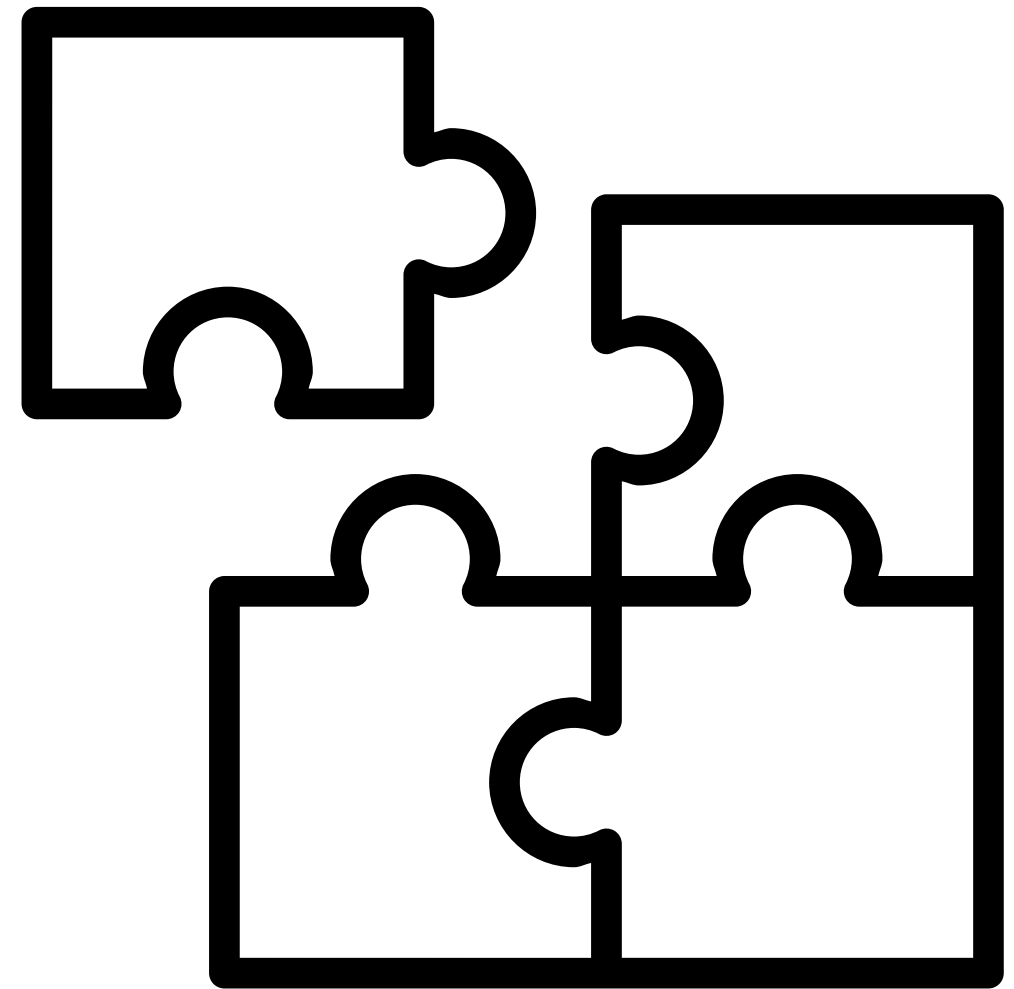
Back



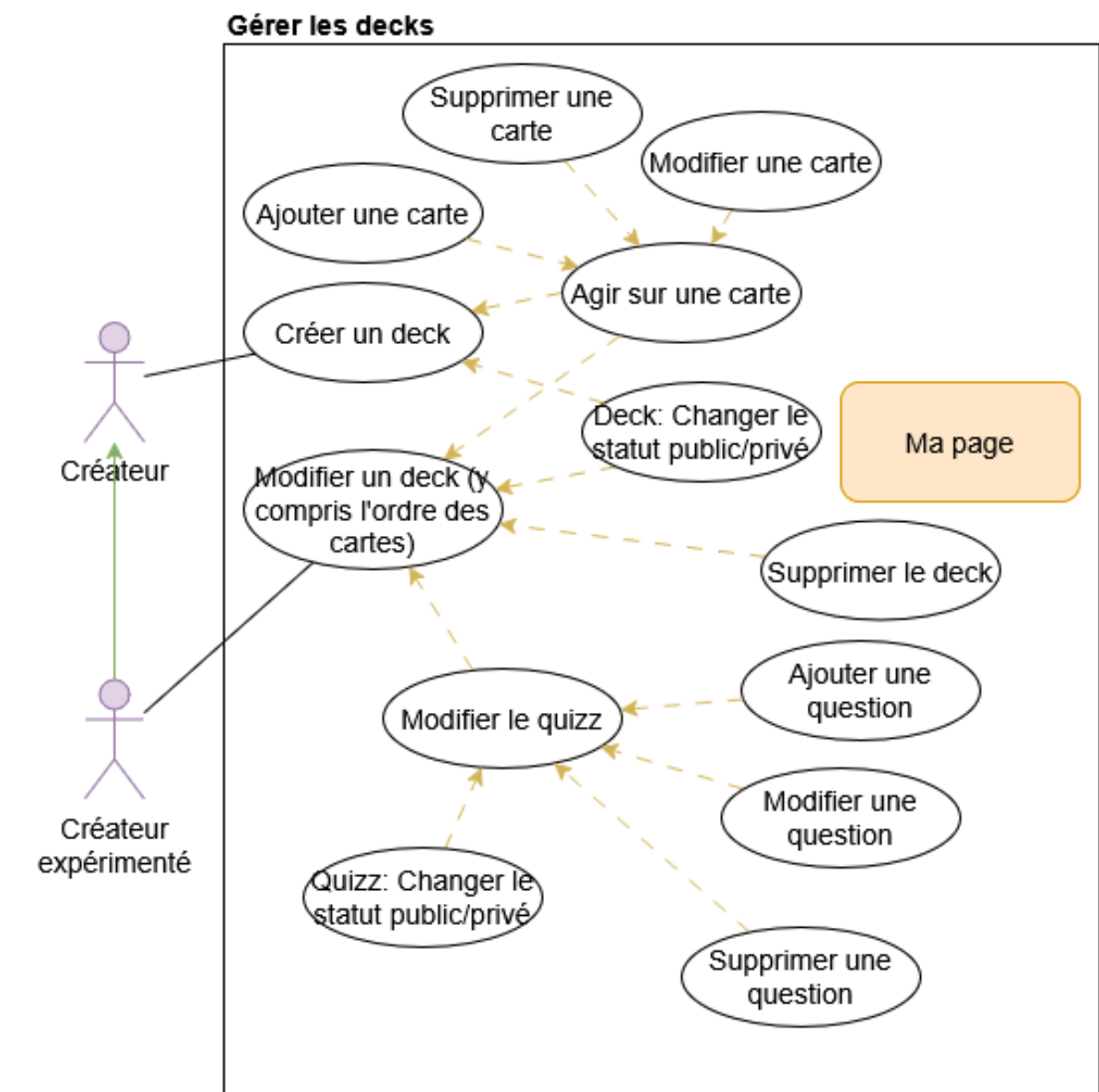
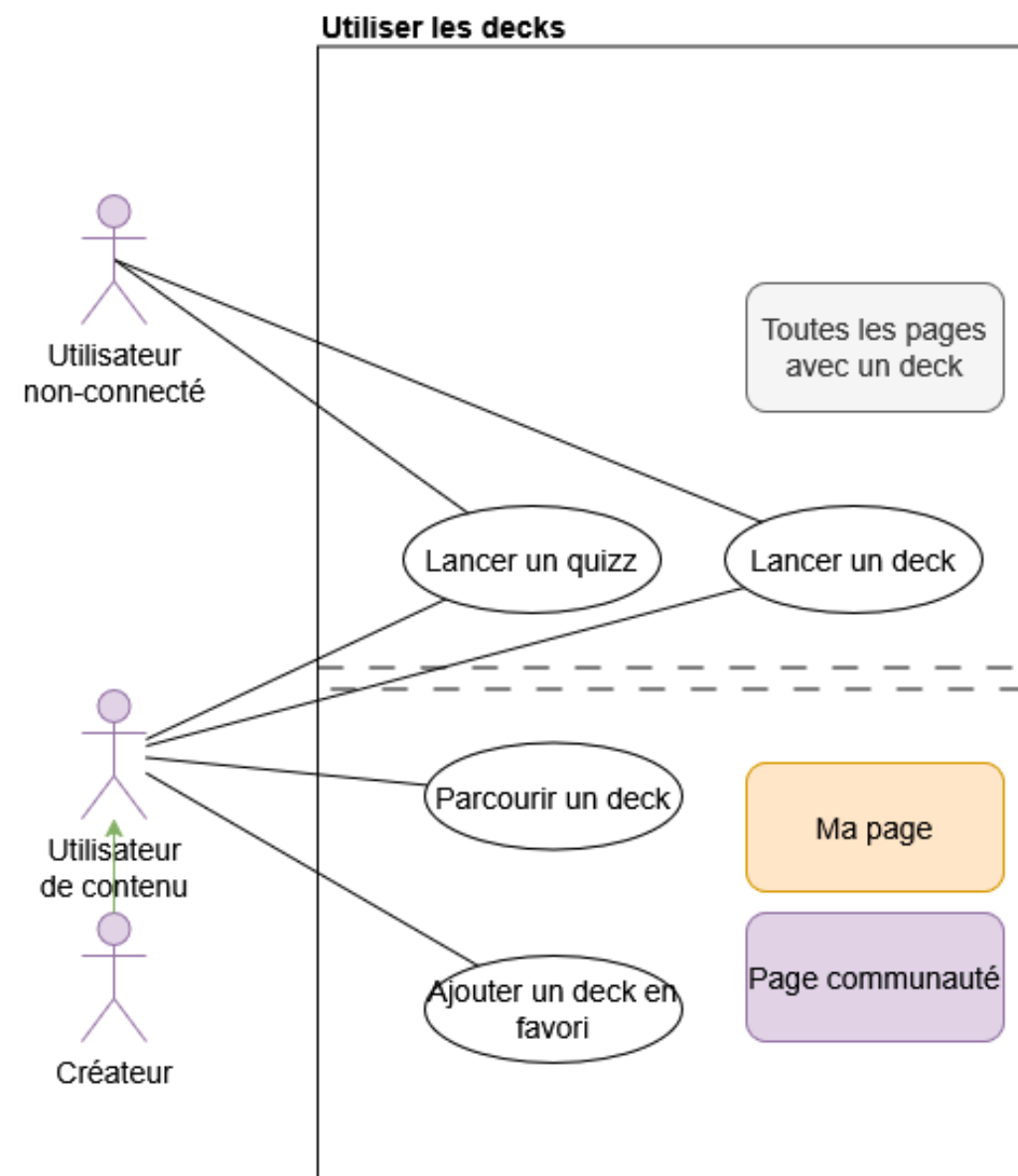
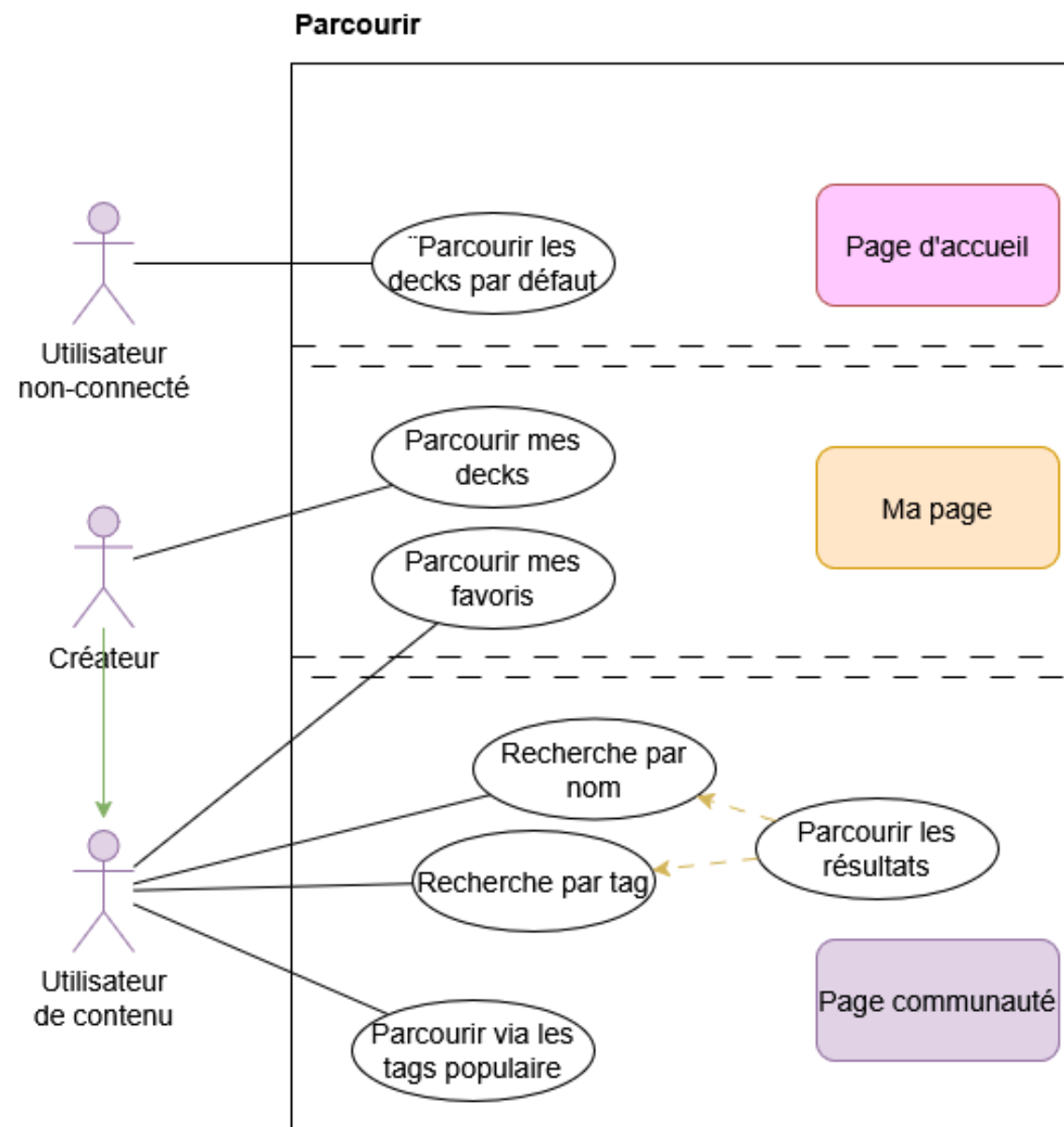
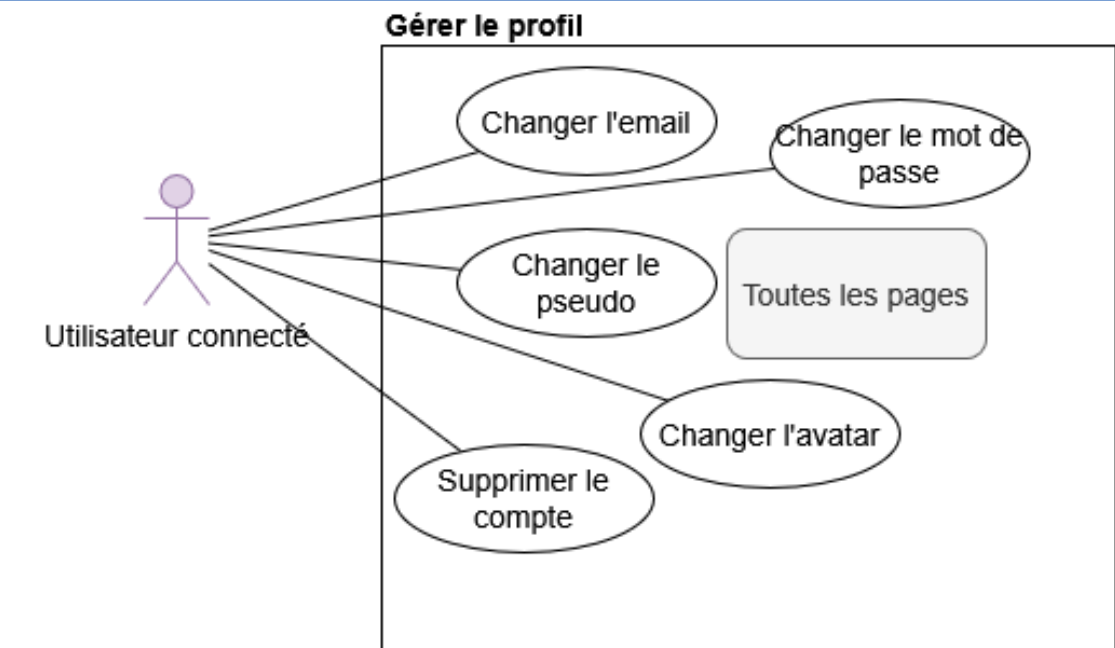
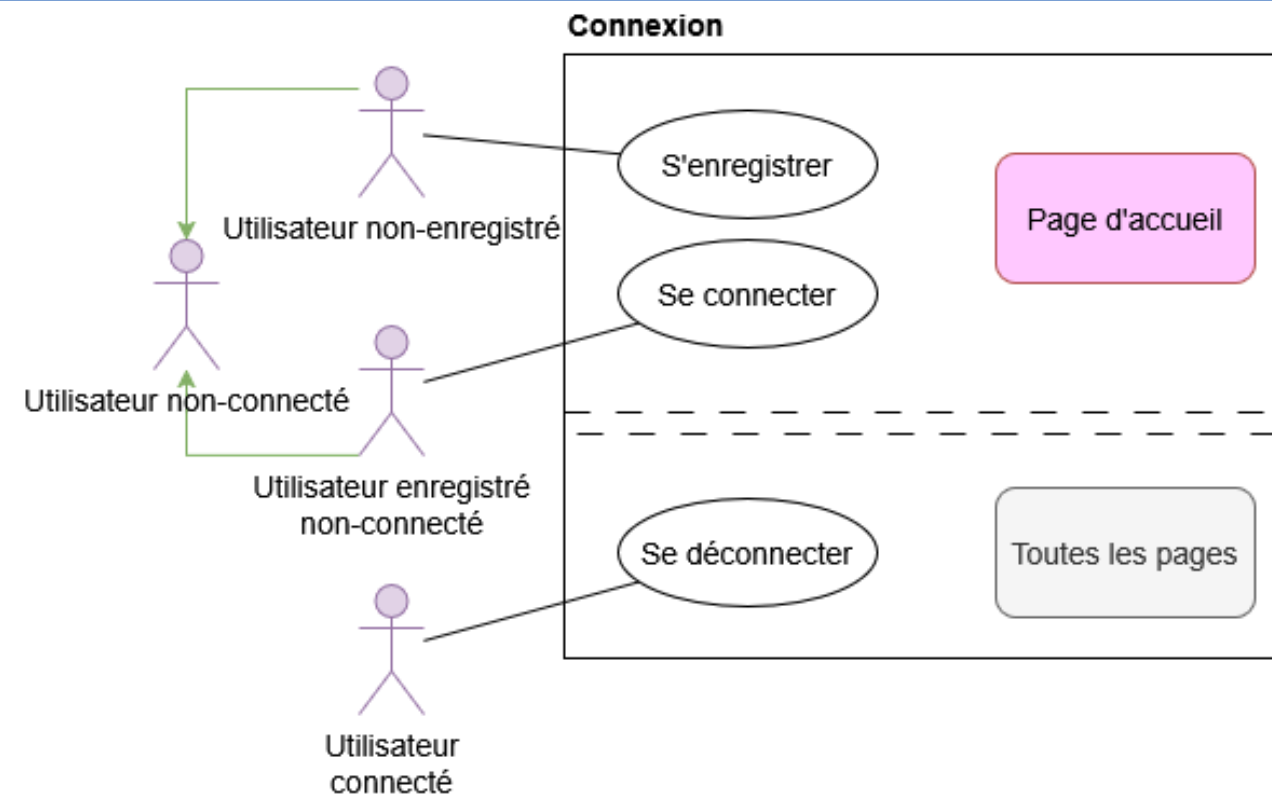
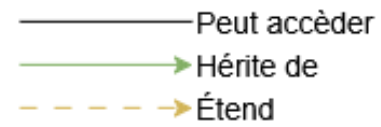
Autre



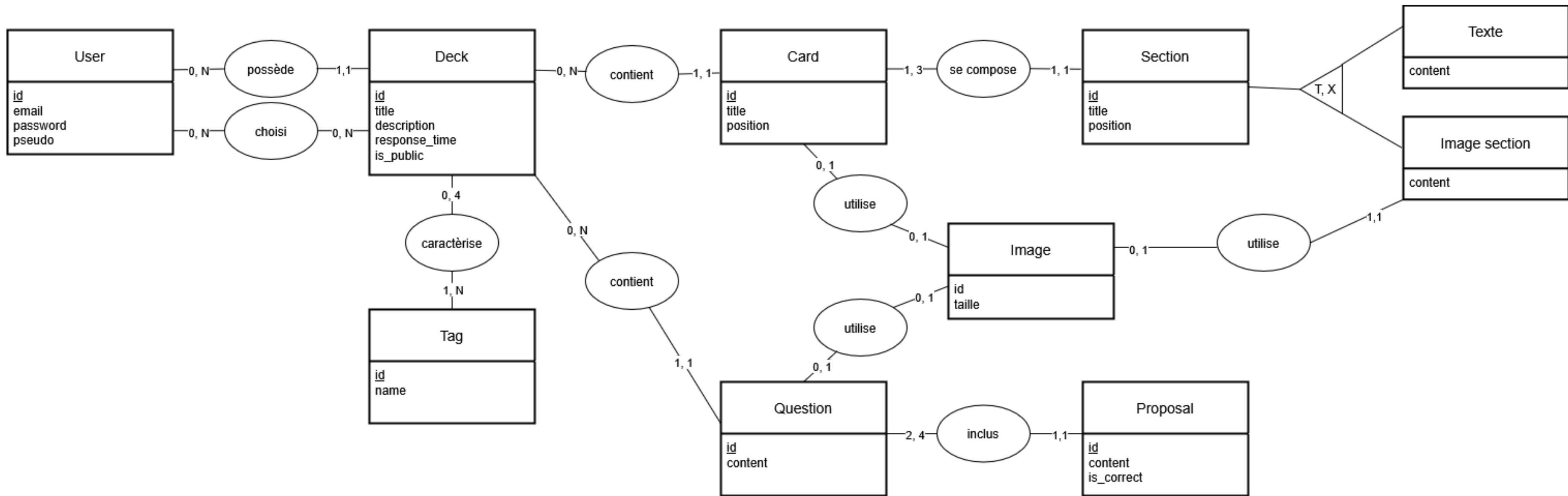
Conception



UML - Diagramme de cas d'utilisation

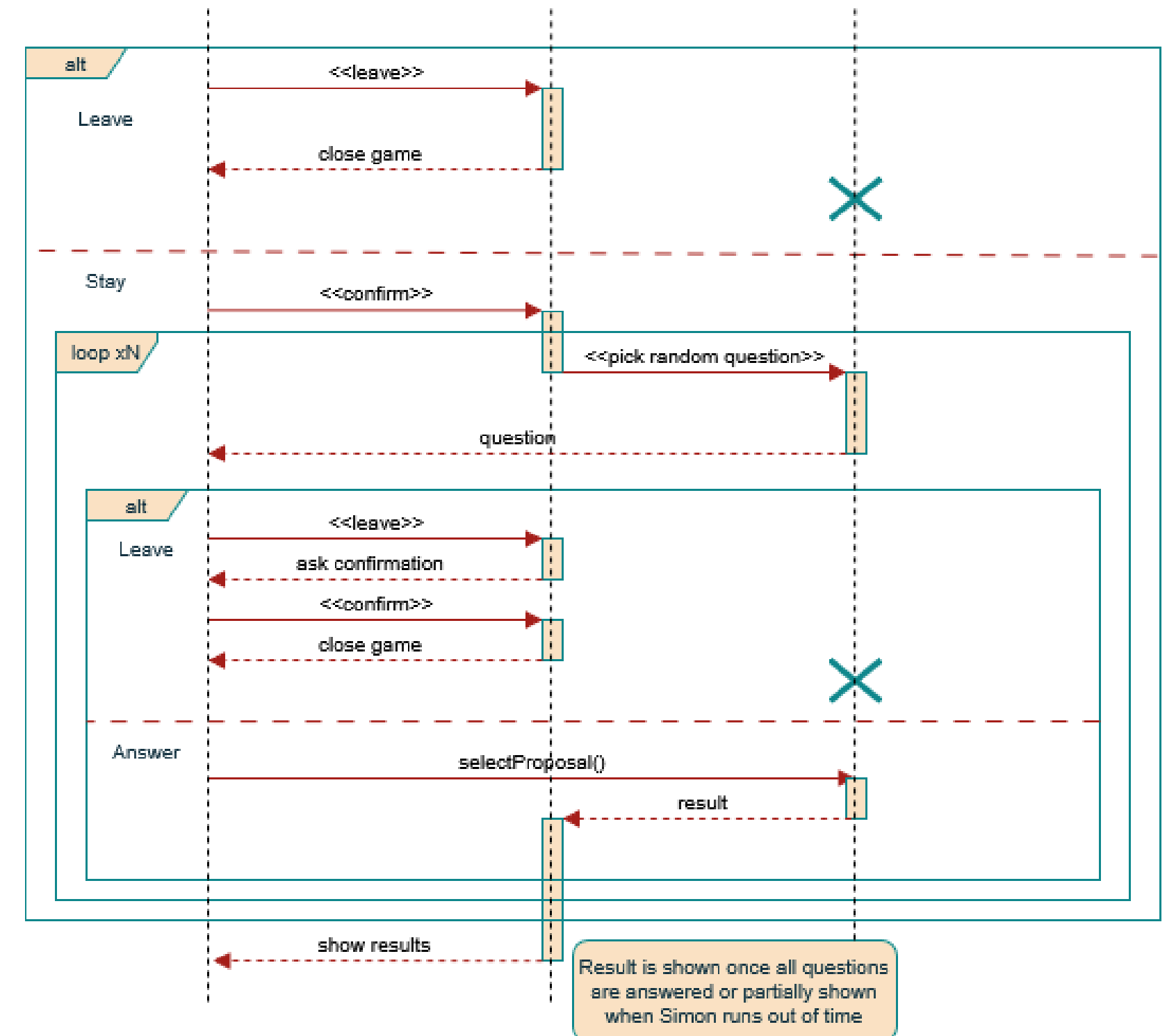
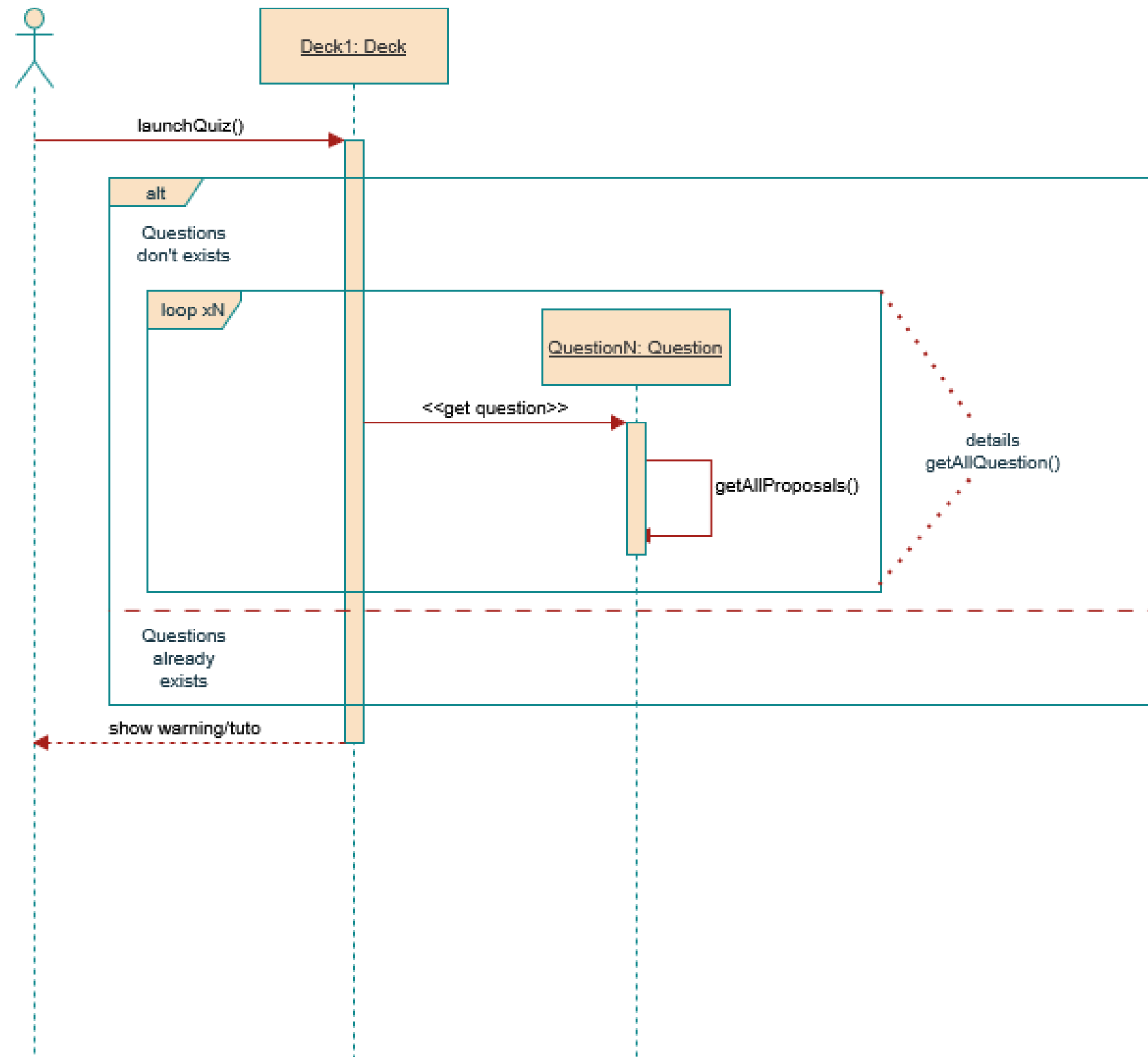


Modèle Conceptuel des Données



UML de séquence

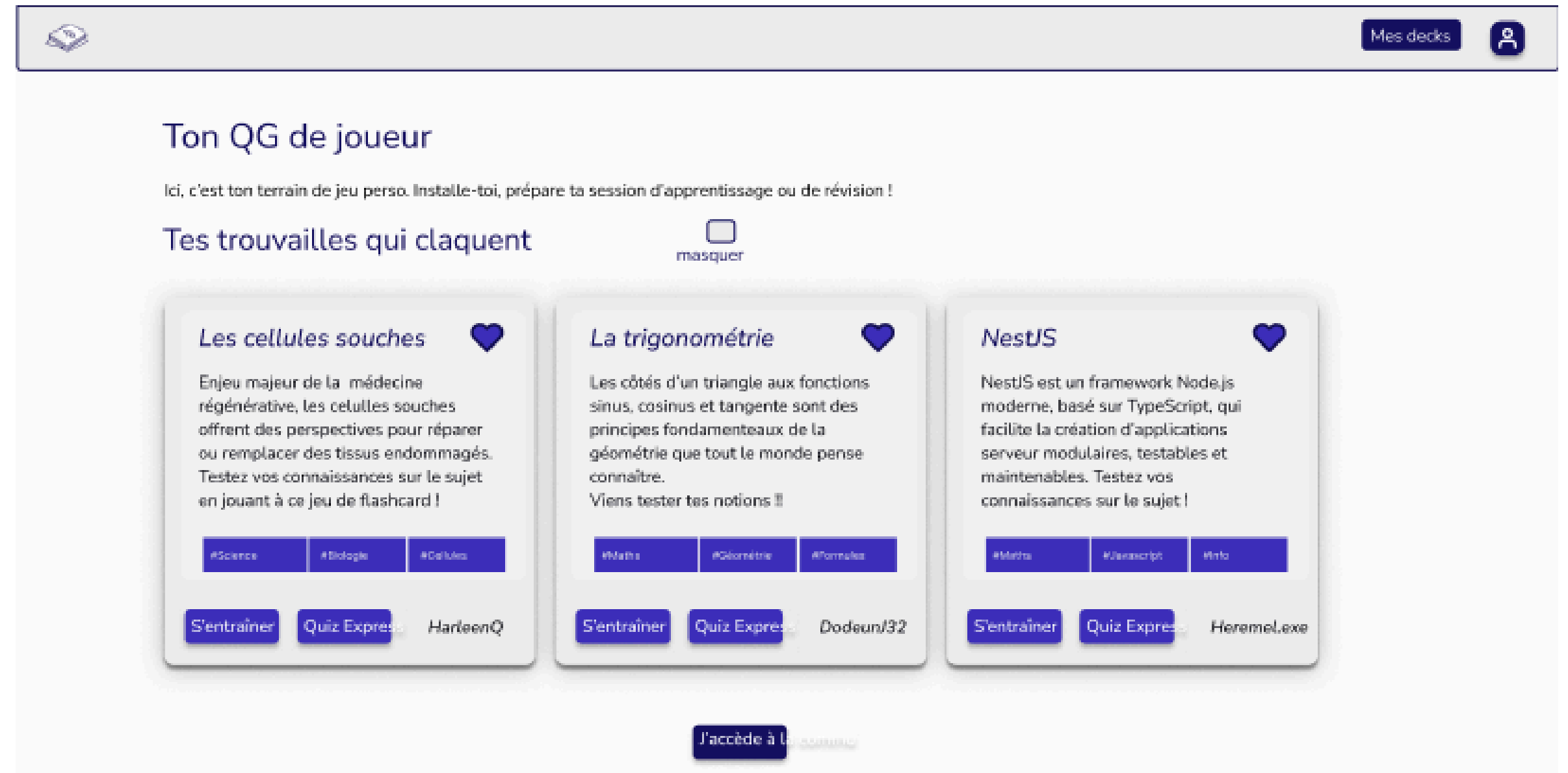
Simon: creator



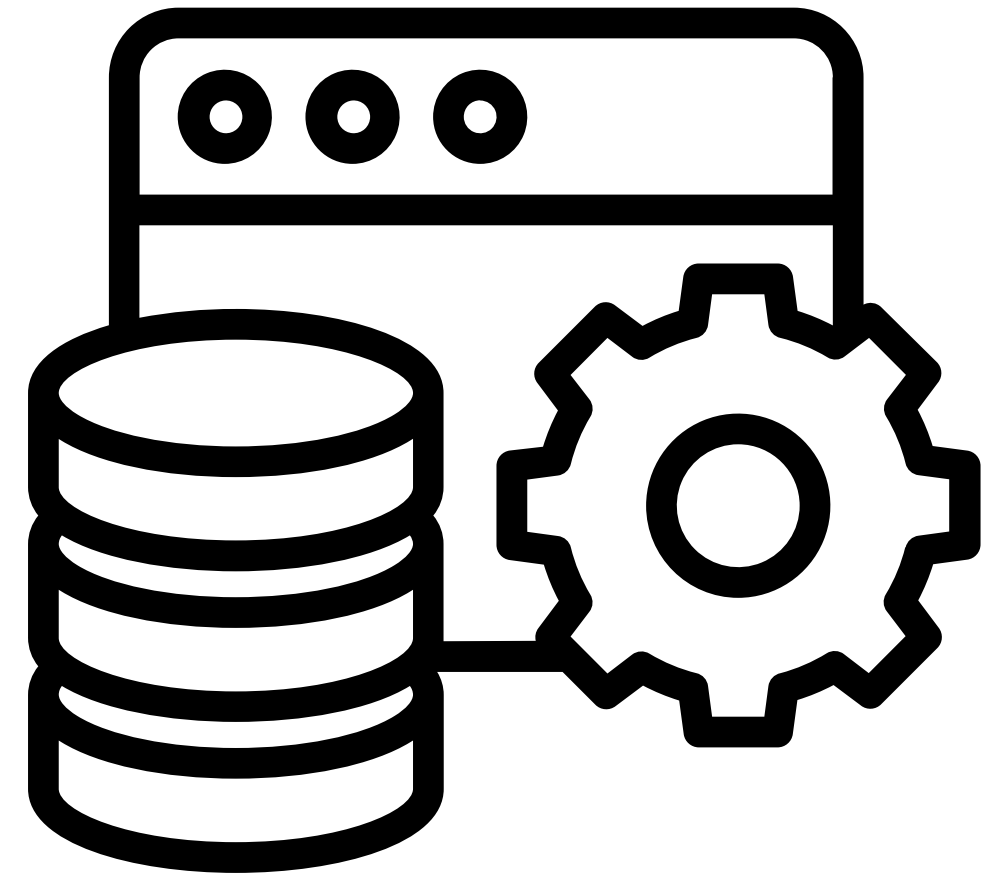
Wireframe - ClientDecks



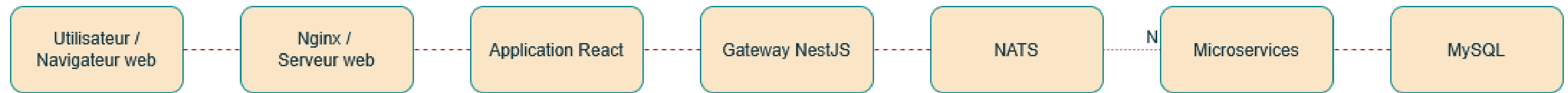
Maquette



Développement back-end



Back: service Quiz



Architecture

Circulation d'une requête

Quiz

Gateway: Controller

```
@Controller("decks/:deck_id/quizzes")
export class QuizController {
    constructor(private readonly gatewayService: QuizService) {}

    // CREATE QUIZ
    @Post()
    @UseGuards(CsrfGuard, AuthGuard)
    createQuiz(@Req() req: Request<AuthGuardPayload>): Promise<CreateQuizResponse> {
        const request = { body: req.body, params: req.params };
        if(!createQuizRequest.validate(request)) {
            throw new BadRequestException();
        }

        return this.gatewayService.createQuiz(
            req.locals.requester,
            request
        );
    }
}
```

Gateway: Service

```
@Injectable()
export class QuizService {
  constructor(
    private readonly quizClient: QuizRpcClient,
    private readonly deckClient: DeckRpcClient
  ) { }

  // CREATE QUIZ
  async createQuiz(
    requester: AuthGuardPayload["requester"],
    request: CreateQuizRequest
  ): Promise<CreateQuizResponse> {
    try {
      await this.assertDeckOwner(request.params.deck_id, requester.id);

      const output = await this.quizClient.createQuiz({
        deck_id: request.params.deck_id,
        quiz: {
          ...request.body.quiz
        }
      });

      return output;
    } catch (error) {
      throw rpcCallExceptionMapper(
        this.rpcCallExceptionMap,
        error
      );
    }
  }
}
```

```
@Injectable()
export class QuizRpcClient {
  constructor(@Inject("RPC_CLIENT") private client: ClientProxy) {}

  async createQuiz(payload: CreateQuizInput): Promise<CreateQuizOutput> {
    return await lastValueFrom<CreateQuizOutput>(  
      this.client  
        .send(QuizRpcAction.Create, payload)  
        .pipe(rpcCallExceptionFilter())
    );
  }
}
```

Microservice: Controller

```
@UseFilters(new RpcExceptionHandler(QuizRpcException.Unexpected))
@Controller()
export class QuizController {
  constructor(private readonly quizService: QuizService) {}

  // CREATE QUIZ
  @MessagePattern(QuizRpcAction.Create)
  async createQuiz(@Payload() payload: CreateQuizInput): Promise<CreateQuizOutput> {
    const quiz = await this.quizService.createQuiz(payload);

    return ({ quiz: quiz.toOutput() });
  }
}
```

```
@Entity()
export class QuizEntity extends QuizFields {
  getProposals(): QuizProposalOutput[] {
    return this.proposals.getItems().map((proposal) => serialize(proposal));
  }

  toOutput(): QuizOutput {
    const quiz = serialize(this);

    return ({
      ...quiz,
      createdAt: quiz.createdAt.toISOString(),
      updatedAt: quiz.updatedAt.toISOString(),
      proposals: this.getProposals()
    });
  }
}
```

Microservice: Service

```
@Injectable()
export class QuizService {
  constructor(
    private entityManager: EntityManager,
    @InjectRepository(QuizEntity)
    private readonly quizRepository: EntityRepository<QuizEntity>
  ) {}

  // CREATE QUIZ
  @EnsureEntityManagerContext()
  async createQuiz(payload: CreateQuizInput): Promise<QuizEntity> {
    const quiz = await this.entityManager.transactional(async (entityManager) => {
      const { proposals, ...quiz } = payload.quiz;

      const quizEntity = entityManager.create(QuizEntity, { ...quiz, deck_id: payload.deck_id });

      const proposalEntities = proposals.map((proposal) => entityManager.create(ProposalEntity, {
...proposal, quiz: quizEntity }));
      quizEntity.proposals.set(proposalEntities);

      await entityManager.persistAndFlush(quizEntity);
      await entityManager.refresh(quizEntity, {
        populate: ["proposals"]
      });

      return quizEntity;
    });

    return quiz;
  }
}
```

ORM: Entités

```
@Entity({ abstract: true })
abstract class QuizFields {
  @PrimaryKey({ type: "uuid" })
  id: string = uuid();

  @Property({ type: "uuid" })
  deck_id: string;

  @Property({ length: 128 })
  question: string = "";

  @Property({ type: "timestamp" })
  createdAt: Date & Opt = new Date();

  @Property({ type: "timestamp", onUpdate: () => new Date() })
  updatedAt: Date & Opt = new Date();

  @OneToMany({
    entity: () => ProposalEntity,
    mappedBy: (e) => e.quiz,
    cascade: [Cascade.ALL],
    orphanRemoval: true,
    hidden: true
  })
  proposals = new Collection<ProposalEntity>(this);

  [HiddenProps]?: "proposals";
}
```

```
@Entity()
export class ProposalEntity {
  @PrimaryKey({ type: "uuid" })
  id: string = uuid();

  @Property({ length: 128 })
  answer: string = "";

  @Property()
  isCorrect: boolean = false;

  @ManyToOne({ entity: () => QuizEntity, hidden: true })
  quiz: QuizEntity;

  [HiddenProps]?: "quiz";
}
```

Traduction SQL

Exemple create dans le dossier projet

```
SELECT
q.id AS quiz_id,
q.question,
q.deck_id,
p.id AS proposal_id,
p.answer,
p.is_correct
FROM quiz AS q
LEFT JOIN proposal AS p
ON p.quiz_id = q.id
WHERE q.deck_id = 'deck_id';
```

Tests unitaires

Service gateway & microservice

```
describe("Get Quiz", () => {
  it("should throw an appropriate exception if the quiz is not found", async () => {
    // Arrange
    const mockRequester = createMockRequester();
    const mockFindDeckByIdOutput = createMockFindDeckByIdOutput();
    const mockGetQuizRequest: GetQuizRequest = {
      params: {
        quiz_id: uuidV7(),
        deck_id: mockFindDeckByIdOutput.deck.id
      }
    };

    deckRpcClient.findDeckById.mockResolvedValueOnce(
      mockFindDeckByIdOutput
    );

    quizRpcClient.findQuizById.mockRejectedValueOnce(
      new RpcCallException(QuizRpcException.NotFound)
    );

    // Act
    const getQuizPromise = quizService.getQuiz(
      mockRequester, mockGetQuizRequest
    );

    // Assert
    await expect(getQuizPromise).rejects.toBeInstanceOf(HttpException);
    await expect(getQuizPromise).rejects.toMatchObject({
      status: HttpStatus.NOT_FOUND
    });
  });
});
```

Tests fonctionnels

Microservice + BDD

```
describe("Create Quiz", () => {
  it("should create a quiz and return the created quiz", async () => {
    const createQuizInput: CreateQuizInput = {
      deck_id: uuidV7(),
      quiz: {
        question: faker.lorem.sentence(6),
        proposals: [
          {
            answer: faker.lorem.sentence(4),
            isCorrect: true
          },
          {
            answer: faker.lorem.sentence(4),
            isCorrect: false
          }
        ]
      }
    };

    const createQuizOutput: CreateQuizOutput = await quizController.createQuiz(
      createQuizInput
    );

    const createdQuizFound = await quizRepository.findOne(
      { id: createQuizOutput.quiz.id },
      { populate: ["proposals"] }
    );
    if(!createdQuizFound) {
      throw new Error("Quiz not found");
    };

    expect(createQuizOutput).toStrictEqual<CreateQuizOutput>({
      quiz: {
        id: createdQuizFound.id,
        deck_id: createdQuizFound.deck_id,
        question: createdQuizFound.question,
        createdAt: createdQuizFound.createdAt.toISOString(),
        updatedAt: createdQuizFound.updatedAt.toISOString(),
        proposals: createdQuizFound.proposals.map((proposal) => ({
          id: proposal.id,
          answer: proposal.answer,
          isCorrect: proposal.isCorrect
        })))
      }
    });
  });
});
```

Développement front-end



ClientDecksPage

Routing

```
{
  path: "/",
  Component: RootPage
},
{
  path: "users/@me/decks",
  Component: ClientDecksPage
},
}
```

```
export function RootPage() {
  const { user } = useClientContext();

  return (
    user
      ? <ClientDecksPage />
      : <LandingPage />
  );
}
```

UseEffect

UseState

UseNavigate

```
export function ClientDecksPage() {
  const [decks, setDecks] = useState<DecksInResponse["decks"]>([]);
  const navigate = useNavigate();
  useEffect(() => {
    async function fetchDecks() {
      try {
        const response = await api.decks.getAll();
        setDecks(response.data.decks);
      } catch {}
    }
    fetchDecks();
  }, []);

  const handleCreate = async () => {
    try {
      const response = await api.decks.create({
        deck: {
          title: "Titre du deck",
          description: "Description",
          response_time: 600,
          is_public: false
        }
      });
      navigate(`/decks/${response.data.deck.id}`);
    } catch {}
  };

  return (
    <>
      <DeckList decks={decks}/>
      <Button variant={"classic"} onClick={handleCreate} className={styles.createDeckButton}>Créer un
nouveau deck</Button>
    </>
  );
}
```

DeckPreview

Props

JSX

Style

```
export interface DeckPreviewProps {
  deck: DeckInResponse["deck"];
  canAccessDeckDetails?: boolean;
};

export function DeckPreview({ deck, canAccessDeckDetails = true }: DeckPreviewProps) {
  const { id, title, description, is_public } = deck;
  const navigate = useNavigate();
  const titleMaxDisplayedLength = 36;
  const descriptionMaxDisplayedLength = 220;

  const handleDeckClick = (e: React.MouseEvent) => {
    if (!canAccessDeckDetails) {
      e.preventDefault();
    }
  };

  return (
    <div className={styles.body}>
      <Link to={`/decks/${id}`} onClick={(e) => handleDeckClick(e)} className={styles.link}>
        <div className={clsx(styles.mainContainer, canAccessDeckDetails && styles.canHover)}>
          <h3>{title.length < titleMaxDisplayedLength ? title : title.slice(0, titleMaxDisplayedLength) +
"..."}</h3>
          <p>
            {description.length < descriptionMaxDisplayedLength ? description : description.slice(0,
descriptionMaxDisplayedLength) + "..."}
          </p>
        </div>
      </Link>
      <div className={styles.container}>
        <Button variant={"classic"} className={styles.createDeckButton} onClick={() =>
navigate(`/decks/${deck.id}/flashcard`)}>S'entraîner</Button>
        <Button variant={"classic"} className={styles.createDeckButton} onClick={() =>
navigate(`/decks/${deck.id}/quiz`)}>Quiz</Button>
        { !is_public && <p>Privé</p> }
      </div>
    </div>
  );
}
```

Tests end to end

Cypress

Utilisateurs:
inscription, connexion, profil, suppression

Deck & Card & Quiz: CRUD

```
describe("deck creation cy test, with cards and quizzes ", () => {  
  it("create a new deck with new cards", () => {  
    // mobile scenario  
    cy.viewport("samsung-s10");  
    // sign-up  
    cy.visit("/");  
    cy.contains("Inscription").click();  
    cy.get("[data-testid=\"pseudo-input\"]").type(goodUser.pseudo);  
    cy.get("[data-testid=\"email-input\"]").type(goodUser.email);  
    cy.get("[data-testid=\"confirm-email-input\"]").type(goodUser.email);  
    cy.get("[data-testid=\"password-input\"]").type(goodUser.correctPassword);  
    cy.get("[data-testid=\"confirm-password-input\"]").type(goodUser.correctPassword);  
    cy.get("[data-testid=\"accept-terms\"]").click();  
    cy.contains("S'inscrire").click();  
    cy.wait(1000);  
    cy.contains("Ton QG de joueur");  
    // new deck  
    cy.contains("Créer un nouveau deck").click();  
    cy.url().should("include", "/decks");  
  });  
});
```

RGPD et SEO

Politique de confidentialité

Conditions générales d'utilisation

Respect de la structure HTML et des balises sémantiques

Rejoins-nous et crée ton deck !

Nom d'utilisateur

Adresse mail

Confirmer l'adresse mail

Mot de passe

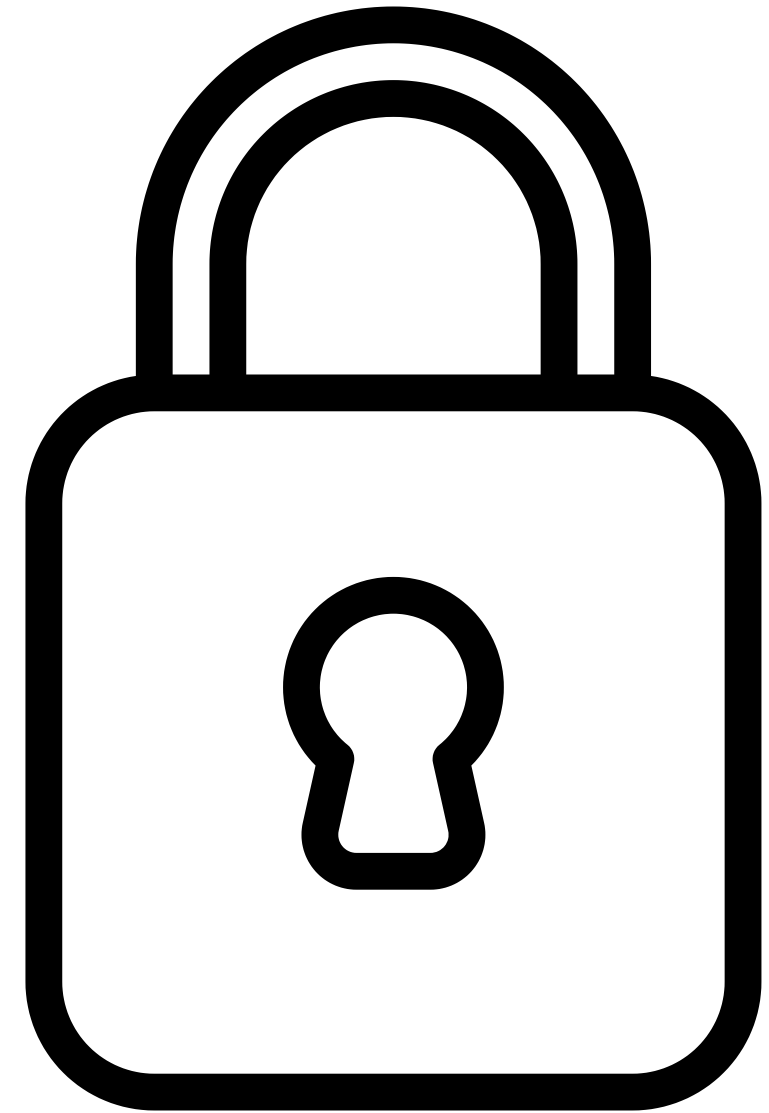
12 caractères minimum, une majuscule, une minuscule, un chiffre, un caractère spécial

Confirmer le mot de passe

☐ J'ai lu et accepte les [conditions générales d'utilisation](#)

S'inscrire

Sécurité



Authentication & Accès

Jason Web Token

Cookies

Guards

Protection des données

Hash & Salt

Données vérifiées avec Valia

ORM

Architecture et maîtrise des erreurs

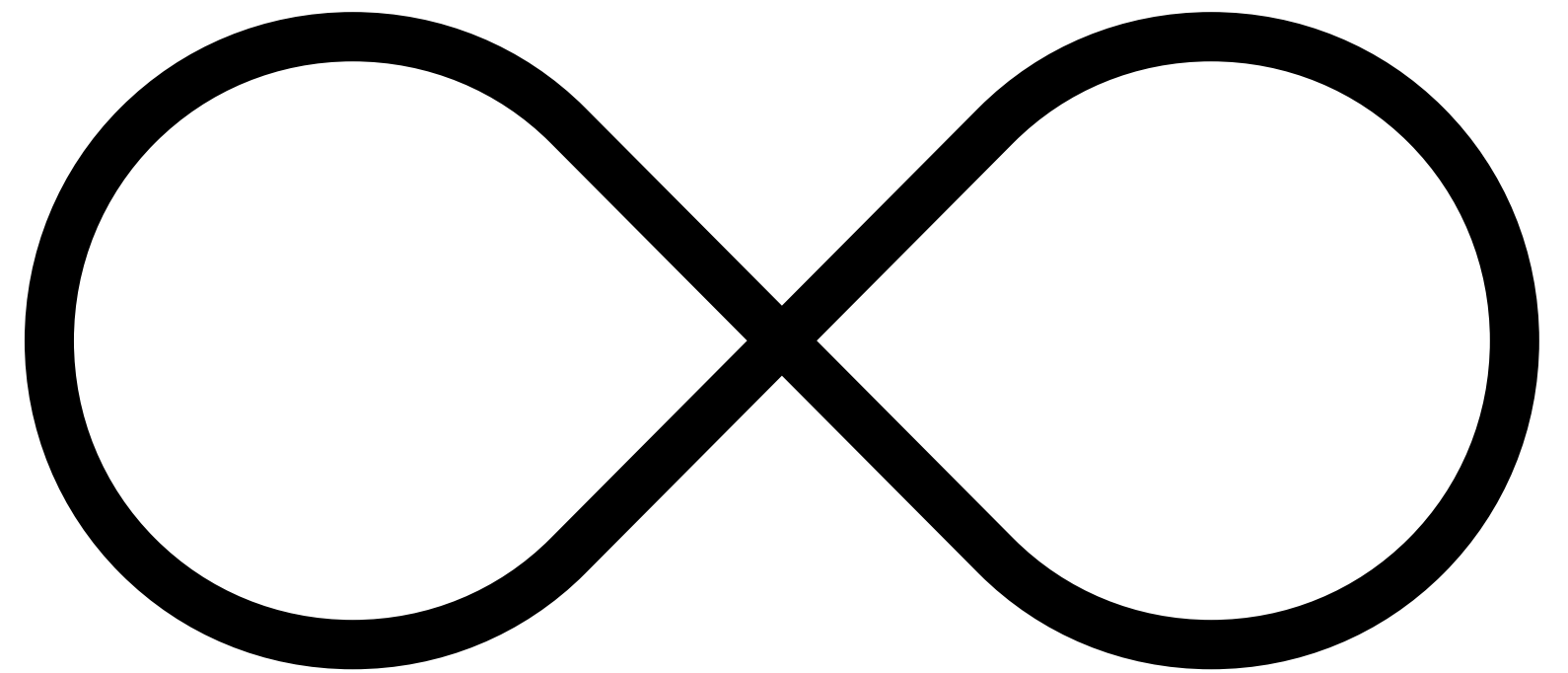
Gateway, point d'entrée unique

Docker networks

Microservice, séparation des responsabilités

Gestion des erreurs

DevOps



Setup serveur

Virtual Private Server

Utilisateur mémoflip

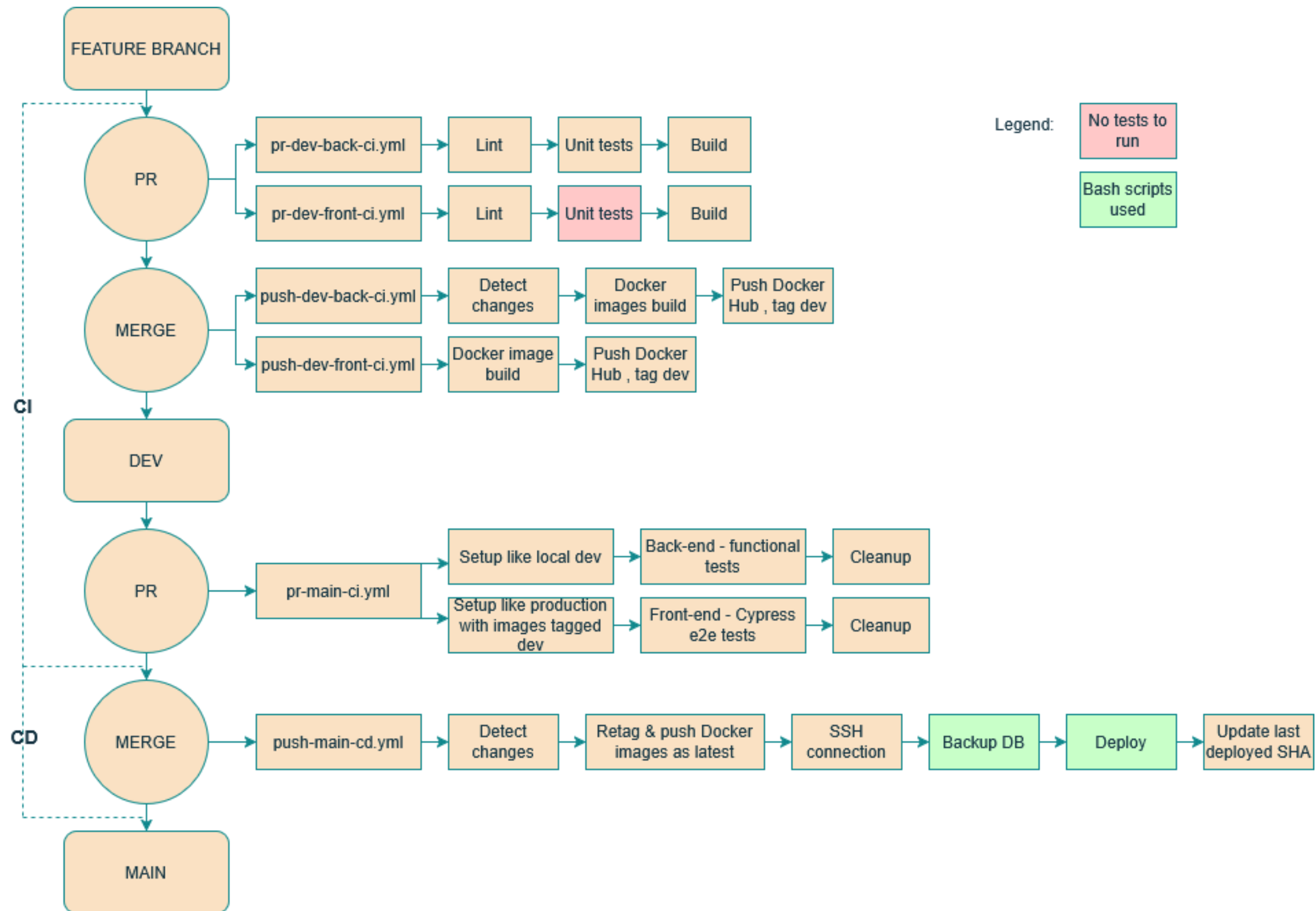
Nginx

Docker

Cron

 all_databases_20260320_020001.sql.gz	Importé par vous • 03:00
 all_databases_20260316_020001.sql.gz	Importé par vous • 16 mars
 all_databases_20260319_020001.sql.gz	Importé par vous • 19 mars

Workflow github



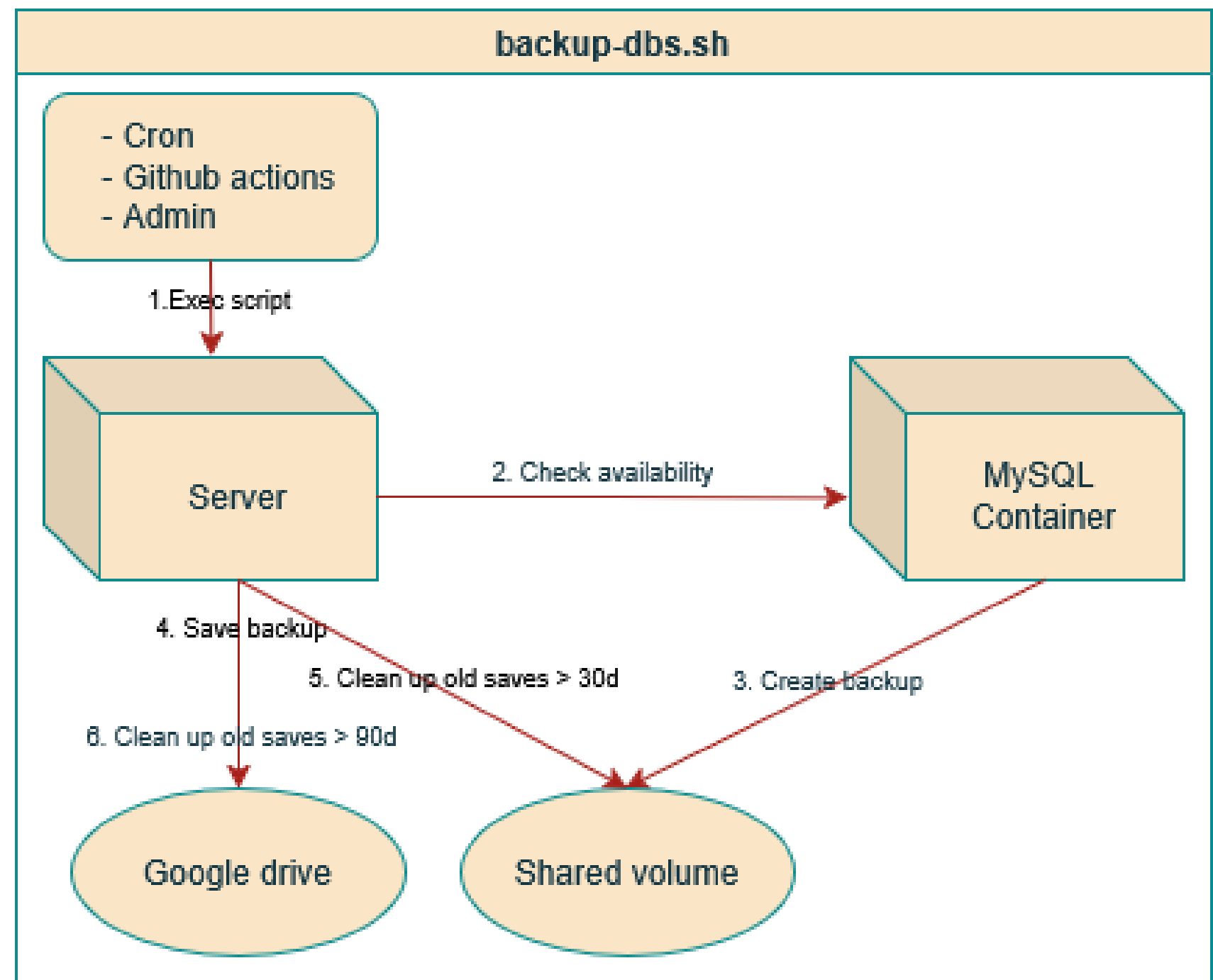
Déploiement continu

SSH

UPDATE sélective

Scripts:

- backup
- deploy
- restore



Évolution du projet

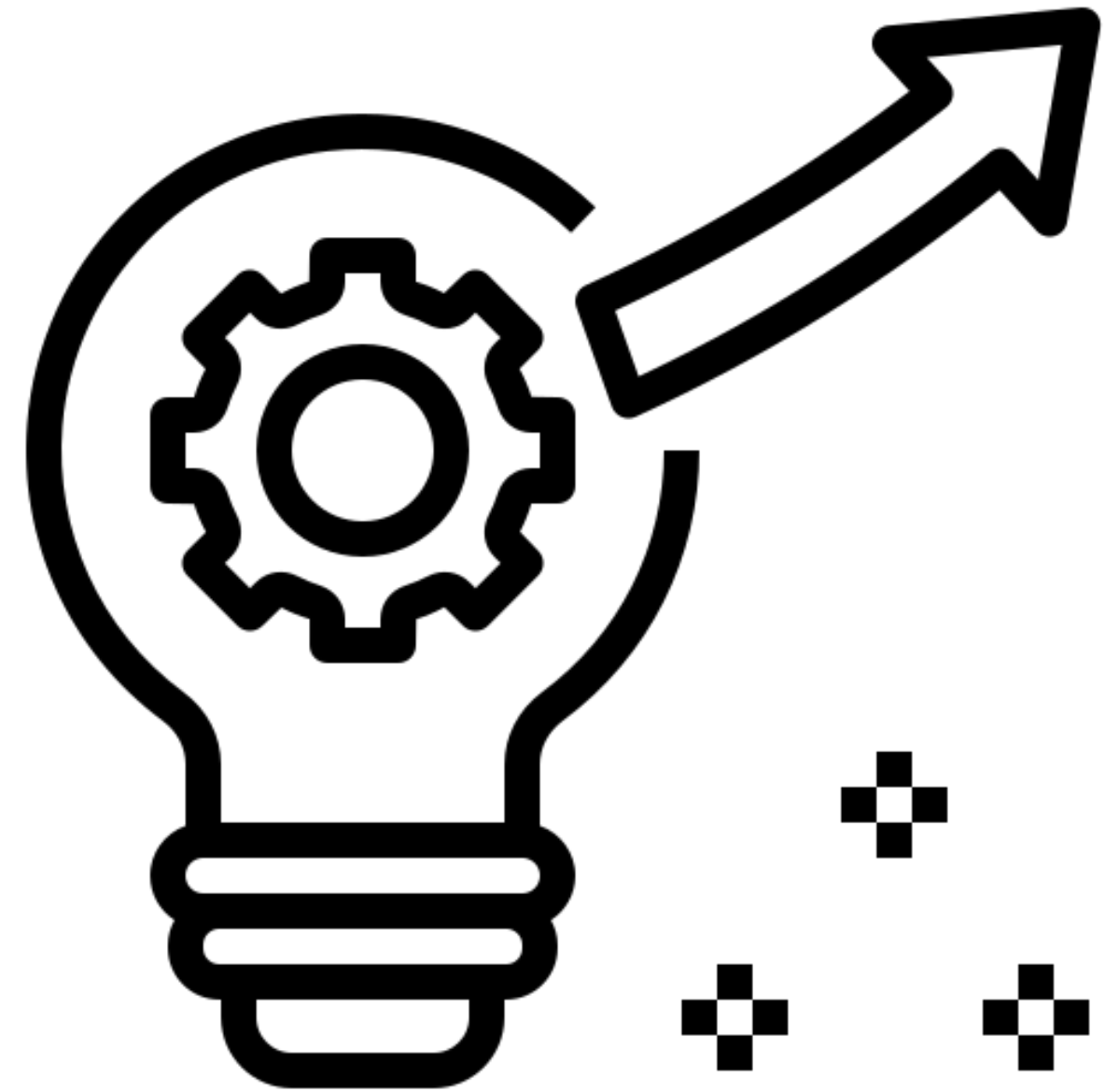
Cache avec Redis

Espace communautaire

Images

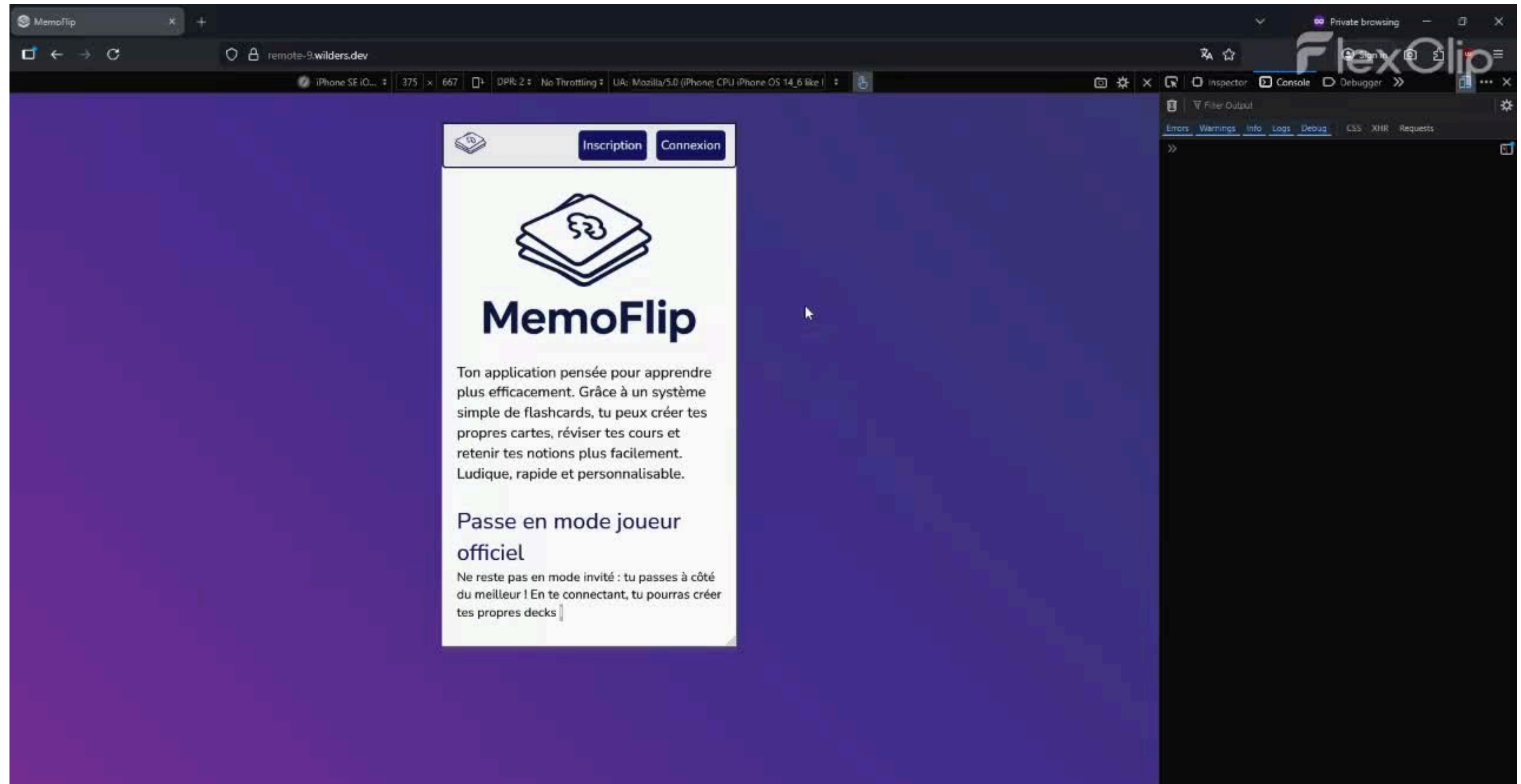
Sécurité renforcée

Scalabilité



Demo

<https://www.youtube.com/watch?v=DjhVt6gf77o>



CONCLUSION

